Bibliotek for å lese og arbeide med OSM-data

Dokumentasjon (Release 1)

Gruppe 8

Navn	Studentnummer
Ole Sander Skjørberg	231433
Mathias Hem	233434
Christian Øyvind Glåmseter	231408
Erling Kristoffer Næsset Arnesen	201404

Emne: Rammeverk og .NET
Dato: 31. mars 2025

Innhold

1 Introduksjon

OsmToolkit er et .NET-bibliotek for lesing, skriving og filtrering av geografiske data fra *OpenStreetMap (OSM)*. Biblioteket fokuserer på støtte for XML-baserte .osm-filer og tilbyr et brukervennlig API for arbeid med noder, veier og relasjoner representert gjennom den interne datamodellen `OsmData`.

I versjon 1 støtter biblioteket følgende funksjonalitet:

- Deserialisering av OSM-data fra XML-fil, streng eller strøm til `OsmData`.
- Serialisering av `OsmData` tilbake til XML.
- Søk etter objekter innenfor en gitt radius, målt i luftlinje fra koordinater.
- Søk etter objekter basert på stiavstand gjennom tilkoblede veier.
- Søk etter objekter basert på en tag.

Alle metoder for lesing og skriving støtter asynkrone operasjoner. Biblioteket har innebygd støtte for logging via `ILogger<T>`, og er utformet for enkel integrasjon med Dependency Injection. Dersom ingen logger injiseres, benyttes automatisk en `NullLogger` for å sikre stabil oppførsel.

OsmToolkit egner seg godt for utviklere som trenger å arbeide med OSM-data i kartapplikasjoner, analyseverktøy eller andre løsninger hvor geografisk informasjon behandles.

2 Installasjon og oppsett

2.1 Steg-for-steg: Installere og bruke biblioteket

`OsmToolkit` distribueres som en NuGet-pakke og kan enkelt legges til i ethvert .NET-prosjekt.

1. Opprett et nytt .NET-prosjekt (for eksempel en konsollapplikasjon eller et API-prosjekt).
2. Installer pakken via NuGet:

```
dotnet add package OsmToolkit
```

3. Bekreft at prosjektet er konfigurert med `.NET 8 SDK`.
4. Registrer tjenestene i Dependency Injection-containeren:

```
using OsmToolKit;
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();
services.AddOsmToolkit();
```

5. Bygg og kjør prosjektet:

```
dotnet build
dotnet run
```

6. Du kan nå bruke bibliotekets funksjonalitet, for eksempel:

```
var deserializer = services.BuildServiceProvider()
            .GetRequiredService<IOsmDeserializer>();

var data = await
   deserializer.DeserializeFromFileAsync("map.osm");
```

2.2 Avhengigheter

- **Krav:**
  - .NET 8 SDK
- **Automatisk avhengighet:** `Microsoft.Extensions.Logging.Abstractions` brukes for intern logging og installeres automatisk som del av NuGet-pakken.
- **Dependency Injection:** Biblioteket er laget med støtte for DI via `IServiceCollection`. Alle nødvendige tjenester registreres med `services.AddOsmToolkit();`.

3 Overordnet struktur

3.1 Modulær oppbygging

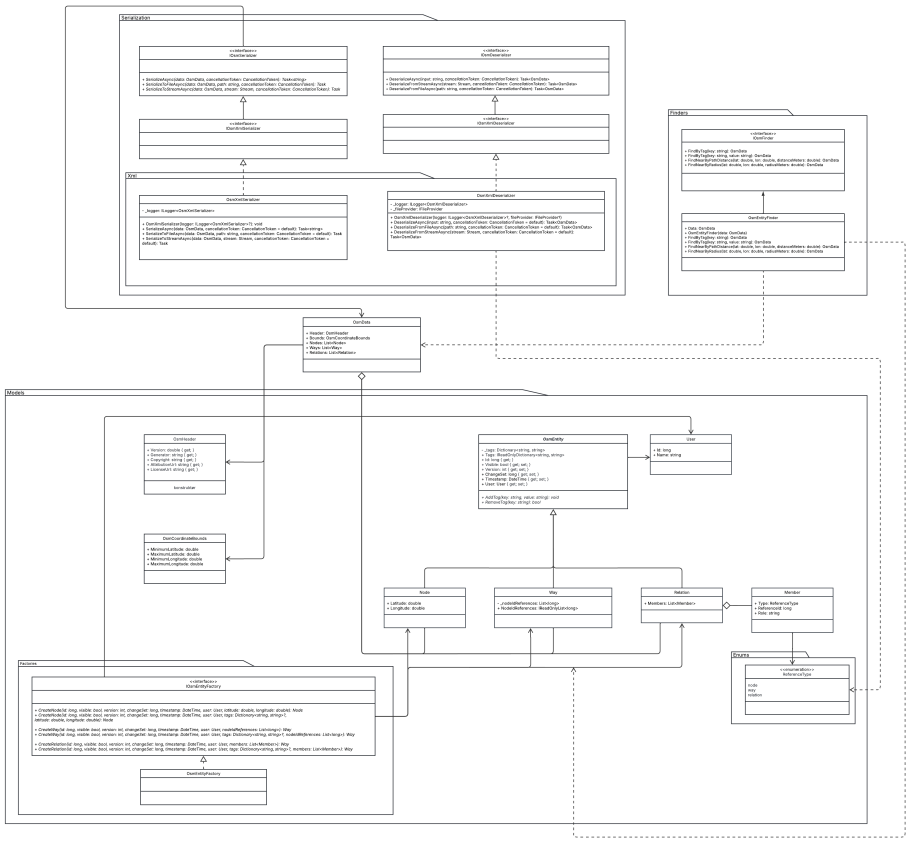
Release 1 av biblioteket organisert i tre hovedmoduler:

- **Modeller:** Inneholder klasser som definerer OSM-data. Eksempler er `OsmEntity`, `Node`, `Way`, `Relation`, samt tilhørende klasser som `User` og enums for `ReferenceType`.
- **Finder:** Inneholder `IOsmFinder` med implementasjon for standard søkefunksjon til å finne `OsmEntitity` instanser med spesifike tags eller innenfor visse rekkevidder.
- **Serializer:** Inneholder grensesnittene `IOsmSerializer` og `IOsmDeserializer` med implementasjoner for XML-basert serialisering og deserialisering.

3.2 Arkitekturdiagrammer

UML-diagram som viser hvordan komponentene henger sammen.
Slik er det strukturert per tid. Det er ikke et ferdig produkt enda.

(Klikk her for å åpne UML-diagrammet i Lucidchart)



Figur 1: Klassediagram for prosjektet

3.3 Designvalg

- **Abstraksjon:** Vi har valgt å bruke en abstrakt klasse `OsmEntity` for å samle felles egenskaper for noder, veier og relasjoner.
- **Grensesnitt:** Bruk av grensesnitt som `IOsmDeserializer` og `IOsmSerializer` sikrer at implementasjonene kan byttes ut og utvides uten å endre det overordnede API.
- **Dependency Injection (DI):** Implementasjonen av `ServiceCollectionExtensions` muliggjør DI, noe som forenkler testing og integrasjon i andre applikasjoner.
- **Enums for typer:** For å sikre enhetlighet og begrense verdiene, brukes enums for `ReferenceType` i Member.

3.4 Feilhåndtering

Feilhåndtering og validering er viktige aspekter av bibliotekets robusthet og pålitelighet. Biblioteket er designet for å håndtere både forventede og uventede feil på en kontrollert måte.

Validering av input

Ved deserialisering fra fil, streng eller strøm, valideres filtyper og innhold før behandling starter:

- Kun `.osm` og `.xml` filtyper støttes i deserialiseringsmetodene.
- Filendelsen sjekkes eksplisitt før forsøk på åpning. Hvis ugyldig, kastes et `ArgumentException`.
- Ved manglende påkrevde XML-attributter i OSM-dataen kastes `InvalidDataException`.
- Generiske parsing-feil fanges med `FormatException`.

Feilhåndtering under IO-operasjoner

Deserialiserings- og serialiseringsmetoder som leser fra eller skriver til fil/stream er omsluttet av trygge `FileStream`- og `StreamReader`-konstruksjoner:

- Feil som `FileNotFoundException`, `IOException`, `UnauthorizedAccessException` og `PathTooLongException` håndteres ved at de lar seg fanges av brukerens kode.
- Metodene benytter seg av asynkron `Stream`-håndtering for å unngå blokkering.
- Det benyttes `CancellationToken` i alle async-metoder for å muliggjøre avbrudd.

Logging av feil

Hvis en `ILogger<T>` blir injisert i konstruktøren til `OsmXmlDeserializer` eller `OsmXmlSerializer`, logges eventuelle feil og diagnostisk informasjon automatisk. Dersom ingen logger blir gitt, benyttes en `NullLogger` slik at systemet forblir stabilt uten ekstern konfigurasjon.

3.5 Logging

Biblioteket benytter strukturert logging via `ILogger`-grensesnittet fra `Microsoft.Extensions.Logging`. Logging hjelper utviklere med feilsøking, ytelsesmåling og forståelse av kontrollflyt i biblioteket.

Formål

Logging benyttes primært under serialisering av OSM-data:

- Når serialisering starter og avsluttes.
- Når entiteter som `Node`, `Way` og `Relation` blir skrevet.
- Ved lagring til fil, strøm eller streng.

Loggingstruktur

Logging er kapslet inn i en internal static partial kalt `SerializationXmlLogMessage`, som inneholder ferdigformatterte meldinger og metoder med bruk av `LoggerMessage`-attributtet for ytelseseffektiv logghåndtering.

- Metodene `LogSerializeAsync`, `LogSerializeToFileAsync` og `LogSerializeToStreamAsync` benyttes for å logge overordnede operasjoner.
- Metodene `LogWritingEntityStarted` og `LogWritingEntityFinished` loggfører når individuelle entiteter skrives.
- Det benyttes `LogLevel.Information` og `LogLevel.Trace` for å skille mellom viktige operasjoner og detaljerte sporingsmeldinger.

Eksempel på loggmelding

```
// Eksempel paa logg under serialisering
[Information] Saved OSM data successfully in output.osm.
[Trace] Started writing Node with ID: '12345'
[Trace] Successfully written Node with ID: '12345'
```

Konfigurasjon

Logger lages gjennom konstruktøren i `OsmXmlSerializer`. Hvis ingen logger er angitt, benyttes `NullLogger` for å unngå nullreferanser og samtidig støtte enkel testing og bruk uten eksplisitt logging.

```
_logger = logger ?? new NullLogger<OsmXmlSerializer>();
```

4 Grunnleggende bruk

Grunnlegende bruk av bibilioteket med noen få eksempler på bruksområder.

4.1 Eksempel på bruk av biblioteket

Under ligger noen eksempler til hvordan man kan bruke de ulike delene av biblioteket, full liste over metoder ligger lenger ned i dokumentet.

Listing 1: Eksempel på bruk av bare deserializer på .osm fil.

```
var services = new ServiceCollection();
services.AddOsmToolkit();

var deserializer = services.BuildServiceProvider()
            .GetRequiredService<IOsmXmlDeserializer>();

var data = await
   deserializer.DeserializeFromFileAsync("map.osm");

foreach (var item in data.Nodes)
{
    Console.WriteLine($"node id:
       '{item.Id.ToString()}'");
    foreach(var tag in item.Tags)
    {
        Console.WriteLine($"Tag: {tag}");
    }
}
```

Listing 2: Eksempel på bruk finder.

```
var finder = services.BuildServiceProvider()
            .GetRequiredService<IOsmFinder<OsmEntity>>();

var result = finder.FindByTag(data, "amenity");

foreach (var node in result.Nodes)
{
    Console.WriteLine($"Resultat: {node.Id},
       [{node.Latitude}, {node.Longitude}]");
}
```

Listing 3: Eksempel på bruk av serializer

```
var serializer = services.BuildServiceProvider()
            .GetRequiredService<IOsmXmlSerializer>();
await serializer.SerializeToFileAsync(data, "map.osm");
```

5 API-referanse

5.1 OsmData Class

Contains lists including all Node, Way and Relation instances registered from OSM-sources.

Constructor	**Description**
OsmData(OsmHeader header, OsmCoordinateBounds bounds)	Initializes a new instance of the OsmData class without any registered instances of OsmEntity. Throws ArgumentNullException if header or bounds is null.
OsmData(OsmHeader header, OsmCoordinateBounds bounds, IEnumerable<Node>? nodes, IEnumerable<Way>? ways, IEnumerable<Relation>? relations)	Initializes a new instance of the OsmData class with registered instances of class types Node, Way and Relation. If any of the input lists are null, empty lists will be assigned instead. Throws ArgumentNullException if header or bounds is null.

Properties	**Description**
Header	OsmHeader describing header details for the OSM-data.
Bounds	OsmCoordinateBounds set for the OSM-data.
Nodes	List of registered Node instances. (Read-only, initialized to empty list if null)
Ways	List of registered Way instances. (Read-only, initialized to empty list if null)
Relations	List of registered Relation instances. (Read-only, initialized to empty list if null)

5.2 Factories

5.2.1 IOsmEntityFactory Interface

Defines factory methods for creating instances of OsmEntity, such as Node, Way or Relation.

`CreateNode(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude)`	Initializes a new instance of the `Node` class with no tags.
`CreateNode(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Node` class with tags.
`CreateWay(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds)`	Initializes a new instance of the `Way` class with no tags.
`CreateWay(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Way` class with tags.
`CreateRelation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<Member> members)`	Initializes a new instance of the `Relation` class with no tags.
`CreateRelation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<Member> members, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Relation` class with tags.

5.2.2 OsmEntityFactory Class

Factory class responsible for creating instances of OsmEntity, such as Node, Way and Relation.

Constructor	Description
`OsmEntityFactory()`	Initializes a new instance of the OsmEntityFactory class.

`CreateNode(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude)`	Initializes a new instance of the `Node` class with no tags.
`CreateNode(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Node` class with tags.
`CreateWay(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds)`	Initializes a new instance of the `Way` class with no tags.
`CreateWay(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Way` class with tags.
`CreateRelation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<Member> members)`	Initializes a new instance of the `Relation` class with no tags.
`CreateRelation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<Member> members, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Relation` class with tags.

5.3 Models

5.3.1 ReferenceType Enum

Contains OsmEntity reference types.

Field	Description
node	Represents members of class type Node.
way	Represents members of class type Way.
relation	Represents members of class type Relation.

5.3.2 User Class

Contains user data about OSM-creators.

Constructor	Description
User(long id, string name)	Initializes a new instance of the User class. Throws ArgumentOutOfRangeException if ID is less than 1. Throws ArgumentException if name is null or empty.

Properties	Description
Id	Unique identifier for User instance.
Name	Name of the user.

5.3.3 OsmEntity Class

Represents a map element.

OsmEntity(long id, bool visible, int version, long changeSet, DateTime timestamp, User user)	Initializes a new instance of the OsmEntity class with no tags. Throws ArgumentOutOfRangeException if id, version, or changeSet is less than 1. Throws ArgumentNullException if user is null.
OsmEntity(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, Dictionary<string, string>? tags = null)	Initializes a new instance of the OsmEntity class with tags. Throws ArgumentOutOfRangeException if id, version, or changeSet is less than 1. Throws ArgumentNullException if user is null.

Methods	Description
AddTag(string key, string value)	Adds or updates a single key-value pair containing metadata for OsmEntity. Throws ArgumentException if key is null or empty.
RemoveTag(string key)	Removes a single key-value pair containing metadata for OsmEntity. Returns true if removal is successful; otherwise, false.

`Tags`	Read-only dictionary of key-value string pairs representing metadata for the `OsmEntity` instance.
`Id`	Unique identifier for `OsmEntity` instance.
`Visible`	Indicates whether the `OsmEntity` instance is currently visible or active.
`Version`	Current version of `OsmEntity` instance.
`ChangeSet`	Id of the set of changes the `OsmEntity` instance was last modified in.
`Timestamp`	Last time the `OsmEntity` instance was changed.
`User`	Creator of `OsmEntity` instance.

5.3.4 Node Class

Represents a point of interest on earth.

Constructor	**Description**
`Node(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude)`	Initializes a new instance of the `Node` class with no tags. Throws `ArgumentOutOfRangeException` if `id`, `version`, `changeSet`, `latitude`, or `longitude` are invalid. Throws `ArgumentNullException` if `user` is null.
`Node(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, double latitude, double longitude, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Node` class with tags. Same validation and exception behavior as the other constructor.

Properties	**Description**
`Latitude`	Distance north or south of equator, within the range of -90 to 90.
`Longitude`	Distance east or west of prime meridian, within the range of -180 to 180.

5.3.5 Way Class

Connection between two or more nodes.

Constructor	Description
`Way(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds)`	Initializes a new instance of the `Way` class with no tags. Throws `ArgumentOutOfRangeException` if `id`, `version`, `changeSet` are less than 1, or if `nodeReferenceIds` has fewer than two elements. Throws `ArgumentNullException` if `user` or `nodeReferenceIds` is null.
`Way(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, List<long> nodeReferenceIds, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Way` class with tags. Same validation and exception behavior as the other constructor.

Properties	Description
`NodeReferenceIds`	Read-only list of reference ids containing the Way instance's nodes.

5.3.6 Member Class

A member within a Relation instance.

Constructor	Description
`Member(ReferenceType type, long referenceId)`	Initializes a new instance of the `Member` class with no role. Throws `ArgumentOutOfRangeException` if `referenceId` is less than or equal to 0.
`Member(ReferenceType type, long referenceId, string? role)`	Initializes a new instance of the `Member` class with a role. Throws `ArgumentOutOfRangeException` if `referenceId` is less than or equal to 0.

Properties	Description
`Type`	The specified OsmEntity type.
`Reference`	A reference to the id of the OsmEntity instance.
`Role`	The specified member role for a Relation instance.

5.3.7 Relation Class

Representation of a geographic feature.

Constructor	Description
`Relation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, IList<Member> members)`	Initializes a new instance of the `Relation` class with no tags. Throws `ArgumentOutOfRangeException` if `id`, `version`, `changeSet` are less than 1, or if `members` has fewer than one element. Throws `ArgumentNullException` if `user` or `members` is null.
`Relation(long id, bool visible, int version, long changeSet, DateTime timestamp, User user, IList<Member> members, Dictionary<string, string>? tags = null)`	Initializes a new instance of the `Relation` class with tags. Same validation and exception behavior as the other constructor.

Properties	Description
`Members`	List of the Relation instance's members.

5.3.8 OsmHeader Class

Represents the header of the OSM-data, describing where it was generated and information about copyright.

Constructor	Description
`OsmHeader(double version, string? generator, string? copyright, string? attributionUrl, string? licenseUrl)`	Initializes a new instance of the `OsmHeader` class. Throws `ArgumentOutOfRangeException` if `version` is less than or equal to 0. If any string argument is `null`, it defaults to an empty string.

Properties	Description
`Version`	Current version number of the OSM schema.
`Generator`	Describes the generator used to create the OSM-data.
`Copyright`	Describes the copyright owners of the OSM-data.
`AttributionUrl`	Contains an URL to the copyright owners' copyright page.
`LicenseUrl`	Contains an URL to the copyright owners' license of use.

5.3.9 OsmCoordinateBounds Class

Represents the latitude and longitude bounds for OSM-data.

Constructor	Description
`OsmCoordinateBounds( double minimumLatitude, double minimumLongitude, double maximumLatitude, double maximumLongitude)`	Initializes a new instance of the OsmCoordinate-Bounds class.

Properties	Description
`MinimumLatitude`	Minimum distance north or south of equator, within the range of -90 to 90.
`MinimumLongitude`	Minimum distance east or west of prime meridian, within the range of -180 to 180.
`MaximumLatitude`	Maximum distance north or south of equator, within the range of -90 to 90.
`MaximumLongitude`	Maximum distance east or west of prime meridian, within the range of -180 to 180.

5.4 Serialization

5.4.1 IOsmSerializer Interface

The `IOsmSerializer` interface defines methods for converting internal `OsmData` instances into OSM data sources compatible with OpenStreetMap standards. This interface allows for serialization directly to a string, to a file on disk, or to a writable stream, making it flexible for various output needs.

Methods	Description
`SerializeAsync(OsmData data, CancellationToken cancellationToken = default)`	Serializes an OsmData instance and returns the XML as a string.
`SerializeToFileAsync(OsmData data, string path, CancellationToken cancellationToken = default)`	Serializes and writes the XML to the specified file path.
`SerializeToStreamAsync(OsmData data, Stream stream, CancellationToken cancellationToken = default)`	Serializes and writes the XML to a provided stream.

5.4.2 IOsmDeserializer Interface

The `IOsmDeserializer` interface provides methods for converting OSM data sources back to the internal `OsmData` instance. These methods allow deserialization from string, file path, or stream, making the deserializer flexible across different I/O contexts.

Methods	Description
`DeserializeAsync(string input, CancellationToken cancellationToken = default)`	Deserializes an OSM XML string into an `OsmData` object.
`DeserializeFromStreamAsync(Stream stream, CancellationToken cancellationToken = default)`	Reads from a stream and deserializes the contents into an `OsmData` object.
`DeserializeFromFileAsync(string path, CancellationToken cancellationToken = default)`	Loads an XML or OSM file from disk and deserializes it into an `OsmData` object.

5.4.3 OsmXmlSerializer

The `OsmXmlSerializer` class is a concrete implementation of the `IOsmXmlSerializer` interface. It provides functionality for converting `OsmData` objects to OpenStreetMap-compatible XML format. The class supports serialization to string, file, or stream.
By default, the class uses an internal logger (`ILogger<OsmXmlSerializer>`) for diagnostics. If none is provided, a `NullLogger` is used.

Constructor	Description
`OsmXmlSerializer(ILogger<OsmXmlSerializer>? logger = null)`	Initializes a new instance of the `OsmXmlSerializer` class. If no logger is provided, a `NullLogger` is used by default for internal diagnostics and debug output.

Methods	Description
`SerializeAsync(OsmData data, CancellationToken cancellationToken = default)`	Serializes the given `OsmData` object to a valid XML string.
`SerializeToFileAsync(OsmData data, string path, CancellationToken cancellationToken = default)`	Serializes the OSM data and writes the XML to the specified file path.
`SerializeToStreamAsync(OsmData data, Stream stream, CancellationToken cancellationToken = default)`	Serializes the OSM data and writes the XML to the provided stream.

5.4.4 OsmXmlDeserializer

The `OsmXmlDeserializer` class implements the `IOsmXmlDeserializer` interface and is responsible for reading and converting XML-based OSM data into structured `OsmData` objects.
It supports deserialization from string, file path, or stream, and includes support for partial or malformed input through proper exception handling. Logging is supported via an optional `ILogger<OsmXmlDeserializer>`.

Constructor	Description
`OsmXmlDeserializer(ILogger<OsmXmlDeserializer>? logger = null, IFileProvider? fileProvider = null)`	Initializes a new instance of `OsmXmlDeserializer` class with optional logging and file access support.

Methods	Description
`DeserializeAsync(string input, CancellationToken cancellationToken = default)`	Parses the provided XML string and converts it into an `OsmData` object.
`DeserializeFromFileAsync(string path, CancellationToken cancellationToken = default)`	Reads a .osm or .xml file from the given path and deserializes it into `OsmData`.
`DeserializeFromStreamAsync(Stream stream, CancellationToken cancellationToken = default)`	Reads from a stream and returns the corresponding `OsmData` object.

5.5 Finders

5.5.1 IOsmFinder Interface

Defines finder methods for finding specific T instances in an OsmData instance.

Methods	Description
`FindByTag(OsmData data, string key)`	Finds T instances that contain a specified tag by only key.
`FindByTag(OsmData data, string key, string? value = null)`	Finds T instances that contain a specified tag by key and value.
`FindNearByPathDistance(OsmData data, double lat, double lon, double distanceMeters)`	Finds T instances from a specified coordinate that are within a specified path distance.
`FindNearByRadius(double lat, double lon, double radiusMeters)`	Finds T instances from a specified coordinate that are within a specified radius.

5.5.2 OsmEntityFinder Class

Used for finding specific OsmEntity instances in an OsmData instance.

Constructor	Description
`OsmEntityFinder(ILogger<OsmEntityFinder>? logger = null)`	Initializes a new instance of the `OsmEntityFinder` class. If no logger is provided, a `NullLogger` is used by default.

Methods	Description
`FindByTag(OsmData data, string key)`	Finds `OsmEntity` instances that contain a specified tag by only key. Throws `ArgumentNullException` if `data` is null, and `ArgumentException` if `key` is null or empty.
`FindByTag(OsmData data, string key, string? value = null)`	Finds `OsmEntity` instances that contain a specified tag by both key and value. Value can be null. Throws the same exceptions as the key-only method.
`FindNearByPathDistance(OsmData data, double lat, double lon, double distanceMeters)`	Finds `OsmEntity` instances within a specified path distance from a given coordinate using Dijkstra's algorithm. Throws `ArgumentNullException` if `data` is null.
`FindNearByRadius(OsmData data, double lat, double lon, double radiusMeters)`	Finds `OsmEntity` instances within a specified radius from a given coordinate. Throws `ArgumentNullException` if `data` is null.

5.6 ServiceCollectionExtension

5.6.1 ServiceCollectionExtensions class

Extends IServiceCollection to register dependencies for using the OsmToolkit.

Method	Description
`AddOsmToolkit(this IServiceCollection services)`	Registers OsmToolkit services into the provided `IServiceCollection`. Adds support for XML serialization, deserialization, and OSM entity finding.

Registered Interface	Implementation and lifetime
`IOsmXmlSerializer`	Registered with implementation `OsmXmlSerializer` using transient lifetime.
`IOsmXmlDeserializer`	Registered with implementation `OsmXmlDeserializer` using transient lifetime.
`IOsmFinder<OsmEntity>`	Registered with implementation `OsmEntityFinder` using transient lifetime.